

Real-Time and Offline Bot Detection System Production Architecture

and Operating Model Date: 2025-08-10 UTC

Real-Time and Offline Bot Detection System Production Architecture

and Operating Model Date: 2025-08-10 UTC

One-Pager Summary

Goal Detect and mitigate bot players in real time during gameplay and offline via periodic analysis, while minimizing false positives, preserving player privacy, and resisting adversarial adaptation. Provide actionable risk scores and enforcement hooks with auditable decisions and strong reliability.

Key Outcomes

Real-time scoring within one hundred milliseconds p95 for live sessions with graduated responses such as soft flags, shadow bans, and hard enforcement. Offline batch detection that surfaces high-confidence accounts, device clusters, and rings within hours. Traceable decisions with reason codes, feature attributions, and replayable evidence for appeals and audits. Multi-tenant isolation by title or region with configurable policies, thresholds, and budgets. Operational excellence with SLOs, canary releases, auto-rollback, and rich observability.

Primary Assumptions

Concurrent players in the low millions globally. Event rate tens of thousands to a few million per second across regions. Models include rules, anomaly detection, supervised models, graph analysis, and device reputation. Data sources include gameplay telemetry, input signals, anti-tamper SDKs, payments or trade, chat, and reports.

Functional Requirements

Ingest player and session events in real time with schema governance and idempotency. Compute online features such as reaction time, aim variance, path entropy, input device cadence, action timing periodicity, and network jitter. Serve risk scores per session, per player, and per device in real time. Emit reason codes and confidence. Trigger policies: warn, soft throttle, match quality penalties, shadow ban, hard ban, or require CAPTCHA or re-authentication. Offline jobs compute long-horizon features, model retraining, graph rings, device linkages, and cohort based drift detection. Admin console for rules, thresholds, rollouts, case review, appeals, and redress. Support experiments and A or B rollouts of models and policies. Export evidence bundles and summary metrics to data warehouse or lake. Provide audit logs, data retention, and deletion workflows.

Non-Functional Requirements

Availability ninety nine point nine five percent for scoring APIs and ninety nine point nine five percent for ingestion. Latency p95 for online score under one hundred milliseconds in-region, p99 under two hundred milliseconds. Batch SLA under three hours for full refresh of offline detections. Durability RPO zero for accepted events and model decisions in hot storage. RTO under fifteen minutes per region. Privacy and compliance SOC two Type two, GDPR, and CCPA with data minimization and user rights. Defense-in-depth against evasion, replay, and tampering.

Architecture Overview

High level Regional cells host ingestion gateways, stream processing, online feature service, model inference, policy engine, evidence store, and short-term state. A thin global control plane manages auth, configuration, policy catalogs, model registry, key management, and cross-region routing. Clients connect to the nearest healthy cell.

Ingestion

Edge gateway terminates TLS, authenticates SDK or server sources with signed tokens, enforces rate limits, validates schemas via registry, and attaches clock skew and jitter metadata. Events are appended to Kafka topics partitioned by tenant id and player id hash to maintain per-player order. Dead letters and quarantine topics capture validation or signature failures for review.

Stream Processing

Flink or Kafka Streams jobs compute online features over sliding windows with watermarks, normalize counters for resets, and attach device and session context. Deduplication uses event id and a short TTL window. Out-of-order handling handled via watermark slack for a few seconds.

Feature vectors and reason signals are written to a low-latency feature store and published to the scoring service.

Online Serving

Feature service fetches recent features by player id or session id from Redis Cluster or a low-latency KV. Model inference service hosts one or more models per title with A or B testing, protected by a sidecar performing input validation and version checks. Policy engine applies rule graphs and thresholds to model outputs and emits decisions, reason codes, and actions. Decisions and features are persisted to a durable store and emitted on a stream for enforcement and analytics.

Offline Analytics and Training

Batch pipelines in Spark or Flink build long-horizon features, train or retrain models, and evaluate performance metrics with backtesting. Graph jobs on ClickHouse or Spark GraphFrames detect rings using shared device ids, IP subnets, payment instruments, trade graphs, and social links. Drift monitors watch feature distributions and label outcomes; automatic gated rollouts when drift is within tolerance.

Storage Strategy

Hot path Redis Cluster for online features and session state with short expiries. Write-through to durable store for critical decisions. Durable path DynamoDB or Scylla for per-player and per-device decisions, reason codes, and evidence pointers. Object storage for raw events and feature snapshots in Parquet with Iceberg or Delta for ACID and time travel. Analytical path ClickHouse or BigQuery or Snowflake for fast slice and dice, cohort analysis, and model evaluation. Maintain gold marts for abuse KPIs and ring casework. Search and cases OpenSearch index for active cases and moderator workflows with full-text notes and tags.

Data Models

Event tenant id, player id pseudonymous, session id, device id, event type, timestamp, attributes json, signature metadata. FeatureVector Online tenant id, player id, session id, feature set id, window, values compact map, generated at. Decision tenant id, player id, device id, session id, model id and version, score, action, confidence, reason codes, issued at, expires at. Evidence decision id, links to event sample ids, feature snapshot id, device fingerprint, network attributes, notes. Policy policy id, rules, thresholds, deciders, rollout cohort, created at, updated at. GraphLink entity a and entity b, link type such as device or ip or payment or social, first seen, last seen, weight.

Latency Budgets and SLAs

Gateway and auth under twenty milliseconds p95. Kafka produce under fifteen milliseconds p95. Streaming feature computation under twenty five milliseconds p95. Feature fetch under five milliseconds p95 from Redis. Model inference under fifteen milliseconds p95 using optimized runtimes. Policy evaluation and serialization under ten milliseconds p95. Aggregate event-to-decision under one hundred milliseconds p95 in-region. Batch pipelines SLA under three hours for complete offline detections. Availability target ninety nine point nine five percent for read and write APIs, scoring, and feature services.

Key Algorithms and Workflows

Feature design Input cadence, variance of aim or turn angles, burstiness, entropy of action sequences, reaction times, movement smoothness, path randomness, device sensor jitter, and network jitter fingerprints. Rules Hard caps on impossible actions per second, inhuman reaction thresholds, aim snap detection, session playtime duty cycle anomalies, inventory or trade abuse thresholds. Anomaly detection Robust z scores per feature and multivariate distance such as Mahalanobis; seasonal baselines by map or mode and skill bracket. Supervised models Gradient boosted trees or small neural nets trained on labeled cases. Calibrated probabilities with isotonic regression. Per-title models with shared feature schema. Graph analysis Connected components, label propagation, and PageRank on device and trade graphs to uncover rings and mules. Use incremental updates with periodic full recompute. Ensemble Score blending of rules, anomaly, and supervised outputs with policy-controlled thresholds and hysteresis to avoid flapping. Adversarial resilience Feature randomization, canary features, and delayed disclosure of reasons. Model stealing mitigation via rate limits and cohort-based perturbations.

Security, Reliability, and Compliance

PII minimization by default. Pseudonymous ids; separate vault for linkage keys where legally required. Encrypt in transit with TLS one point three and at rest with KMS; optional customer-managed keys. RBAC and ABAC for console and APIs. Tamper-evident logs. Regional cells for residency; multi-AZ for every tier. Disaster recovery with tested backups and object storage versioning. Signed SDK payloads and attestation from anti-tamper modules when available.

Multi-Tenancy

Tenant id carried in every key. Namespaces and quotas per title or region. Optional hard isolation with dedicated cells and VPCs for premium tenants. Per-tenant policies, models, thresholds, and dashboards. Usage-based rate limits and spend guardrails.

CI and CD and MLOps

Everything as code with Terraform or Pulumi and Helm. Model registry with lineage and rollbacks. Argo Rollouts for canary and progressive delivery gated on SLOs, precision or recall on shadow traffic, and error budgets. Feature definitions in code and tested. Online feature compat checks enforced before model deploys. Data contracts validated in CI. Blue or green deploys for inference with shadow evaluation before cutover.

APIs and Integrations

Ingest API for telemetry and decisions with idempotency keys. Score API returns risk score, action, and reasons. Batch Score API for offline replays. Policy API manages rules and rollouts. Case API supports review, notes, and appeals. Export and evidence APIs package events, features, and decisions for audits. Webhooks and streams for enforcement hooks to matchmaking, economy, chat, and moderation tools.

Observability and Operations

OpenTelemetry for end-to-end tracing. Golden dashboards for ingestion lag, feature freshness, inference p95 and p99, policy decisions per action, false positive rate estimates, drift metrics, and ring graph growth. SLO-aware autoscaling of gateways, streaming jobs, Redis shards, and inference replicas. Canaries monitored with precision and recall against labeled holdouts. Runbooks for drift, hot shards, cache misses, backlog, and regional failover.

Capacity Planning and Cost Estimates

Reference busy region with two hundred thousand events per second sustained and peaks at one million. Kafka six to nine brokers, replication three, partitions sized to match parallelism. Flink parallelism sixty four to one hundred twenty eight; checkpoint intervals under five

seconds. Redis twelve shards or more with memory headroom under sixty five percent. Inference autoscaling targets p95 under fifteen milliseconds; model servers on GPU only if model complexity warrants. Durable KV sized for thousands of writes per second and hot reads. Object storage costs scale with retention. Rough monthly cost order of magnitude thirty thousand to ninety thousand per busy region depending on retention and model complexity.

Testing Strategy and Failure Drills

Unit tests for feature math, rules, and policy graphs. Property tests for idempotency and ordering. Contract tests for SDK and ingest schemas. Load tests for hot shards and bursty sessions. Soak tests for memory fragmentation and long-horizon features. Chaos drills broker loss, Redis shard failover, inference pool churn, and regional isolation. Red team adversarial tests that try evasion, spoofing, and model drift. Disaster recovery drills for object storage and durable KV.

Phased Build Plan

MVP three months single region cell, online rules and features for core modes, Redis feature cache, basic supervised model, policy engine, and Score API with soft actions. Case console minimal. Phase two six to nine months multi-region cells, graph linkage and ring detection, appeals workflow, evidence bundles, A or B testing and shadow evaluation, and drift monitors. Stronger moderation integration. Phase three twelve months hard isolation options per tenant, advanced models per mode and skill bracket, adversarial defenses, cross-title ring detection, and stricter SLOs for premium titles.

Key Choices, Trade-Offs, Risks, and Mitigations

Real-time features and models deliver immediate protection but can raise false positives; mitigated with hysteresis, soft actions, and offline confirmation. Graph methods surface rings but are compute heavy; mitigated with incremental updates and sampling. Kafka and Flink add ops cost but provide replay and deterministic pipelines; mitigated by managed services and IaC. Storing rich evidence aids audits but increases privacy risks; mitigated with minimization, retention limits, and access controls. Attackers adapt; continuous model iteration, red teaming, and fast rollbacks are essential.